Principles of Programming Languages

Draft Proposal: ClariFix Programming Language

COSC 303: Programming Language Design

Course:	COSC 303 – Programming Language Design
Section:	BSCS 3-3
Language Name:	***ClariFix***
Group Members:	***Balayanto**, Czarina Mae M.* ***Brase**, Mark James B.* ***Castillo**, Marc Lowel J.* ***Derecho**, Jimson Aldrich G.* ***Hipolito**, Brandon Boyd M.* ***Intong**, Sean Gabriel M.* ***Regencia**, Hannah Samantha C..*

I. Introduction

ClariFix is a procedural, dynamically typed, interpreted programming language designed for first-time programmers. Its distinguishing feature is a structured error-reporting system: every error is reported with its location, a plain-language explanation of what went wrong and why, and a concrete suggestion for fixing it. The main problem ClariFix addresses is the difficulty beginners face in understanding and troubleshooting programming errors.

The language keeps Python's readable keywords (set, show, ask). so that statements read like short English sentences. Each simple statement ends with a period (.), like a sentence. It supports variables, six primitive data types, operators, conditional statements, loops, and functions. ClariFix intentionally omits classes, modules, and file I/O to keep the learning surface small.

Target users: students in their first programming course.

Paradigm: procedural.

Typing: dynamic and strong (no implicit conversion between text and numbers).

Execution model: interpreted. The source passes through a lexer, a parser, a semantic checker, and then execution. Each stage reports its own class of errors (see the Error System section).

Out of scope: classes and inheritance, modules, file and network I/O, concurrency.

II. Inspiration & Influences

ClariFix takes inspiration from Python's simple, readable, and beginner-friendly syntax (van Rossum et al.). Python's straightforward structure lets beginners focus on concepts rather than punctuation, which influenced ClariFix's simple keywords, indentation-based blocks, variables, conditions, loops, and functions.

ClariFix targets an area where beginners still struggle in Python: interpreting errors. Python's messages are useful, but many still assume prior knowledge of terms such as "traceback", "NoneType" or "unsupported operand". Newer Python versions add some suggestions, but the format is not uniform across error kinds. ClariFix makes one consistent four-part format (what, where, why, how to fix) the core of the language rather than an add-on. This follows research showing that raw compiler messages are often hard for novices to act on (Marceau et al., 2011; Becker et al., 2019).

The noise-word idea is influenced by COBOL-style and sentence-like DSLs, where optional filler words make code read as natural language without changing its meaning.

III. Syntactic Elements of the Language

1. Character Set

Let:

$$\Sigma = \text{LETTERS} \cup \text{NUMBERS} \cup \text{SYMBOL} \cup \text{WHITESPACES}$$

Where:

$$\text{LETTERS} = \{\text{LETTERS}_{\text{lower}} \cup \text{LETTERS}_{\text{upper}}\}$$

$\text{LETTERS}_{\text{lower}}$ = {a, b, c, d, e, f, g, h, i, j, k, l, m, n, o, p, q, r, s, t, u, v, w, x, y, z}

$\text{LETTERS}_{\text{upper}}$ = {A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, P, Q, R, S, T, U, V, W, X, Y, Z}

NUMBERS = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9}

SYMBOL = {(,), [,], ', ", !, =, +, -, *, /, \, %, :, ,, ., <, >, ^, #, _}

WHITESPACES = {space, /t, /n}

The two-character sequences ==, !=, <=, >=, //, /* and */ are formed from these symbols. Other characters may appear inside string literals and comments but nowhere else. A stray character such as @ or $ outside a string or comment causes error E103.

2. Identifiers

This rule dictates that an identifier must:

1. Begin with a letter (A-Z, a-z) or an underscore (_).
2. Follow the initial character with any combination of letters, digits (0-9), or underscores.
3. Contain no spaces or other symbols.
4. Have a maximum length of 32 characters. A longer name causes error E104.
5. Be written with the correct case, because identifiers are case-sensitive and uppercase and lowercase letters are treated as distinct characters.
6. Differ from every keyword, reserved word, and noise word. Using one causes error E202.

Examples: studentName, _score1, totalAmount.

3. Operations Symbols and Precedence

Arithmetic

Operator	Description
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulo
^	Exponentiation
++	Increment
—	Decrement

Logical

Operator	Description

&&	AND
||	OR
!	NOT

Relational

Operator	Description
==	Equal to
!=	Not Equal to
<	Less than
>	Greater than
<=	Less than or Equal
>=	Greater than or Equal

Assignment

Operator	Description
= (assign)	Simple assignment (stores a value)
+= (Add and assign)	Adds the right value to the variable and stores the result. - *Example: x += 3 (Equivalent to: x = x + 3)*
-= (Subtract and assign)	Subtracts the right value from the variable and stores the result. - *Example: x -= 4 (Equivalent to: x = x - 4)*
*= (Multiply and assign)	Multiplies the variable by the right value and stores the result. - *Example: x *= 5 (Equivalent to: x = x * 5)*

/= (Divide and assign)	Divides the variable by the right value and stores the result. - *Example: x /= 3 (Equivalent to: x = x / 3)*
%= (Modulo and assign)	Divides the variable by the right value and stores the remainder. - *Example: x %= 5 (Equivalent to: x = x % 5)*
^= (Exponentiate and assign)	Raises the variable to the power of the right value and stores the result. - *Example: x ^= 3 (Equivalent to: x = x ^ 3)*

Precedence from highest to lowest:

Level	Category	Operators	Associativity
1	Grouping/ Logical NOT / Increment/ Decrement	(), ++, --, !,	left to right
2	Exponentiation	^	right to left
3	Multiplication /Division /Modulo	*, /, %	right to left
4	Addition / Subtraction	+, -	left to right
5	Relational	<, >, <=, >=	left to right
6	Equality	==, !=	left to right
7	Logical AND	&&	left to right
8	Logical OR	||	right to left

4. Keywords and Reserve Words

Keyword and Reserve Words	Python Equivalent	Explanation
set	=	Assigns a value to a variable.
show	print()	Displays output.
ask	input()	Gets input from the user.
if	if	Checks a condition.
elif	elif	Checks another condition.
else	else	Executes when previous conditions are false.
while	while	Repeats while a condition is true.
for	for	Iterates over a sequence or range.
in	in	Used in loops and membership checks.
function	def	Defines a function.
return	return	Returns a value from a function.
break	break	Exits a loop.
continue	continue	Skips to the next loop iteration.

5. Noise Words

The noise set is explicitly defined as N = { to, the, of, then }. The lexer drops these tokens in the positions listed below, so they never reach the parser. They are reserved, so none can be used as a variable name.

Noise word	Where allowed	Example
to	After the identifier in set	set total to 10 is the same as set total = 10
then	Between a condition and its colon	if x > 5 then: is the same as if x > 5:
the, of	Immediately before an identifier or expression in set, show, ask	show the total is the same as show total

6. Comments

Type	Syntax	Example
Single-line	# to end of line	# this calculates the total
Multi-line	/* ... */	/* Calculates the class average */

Comments are ignored during execution. An unclosed /* causes error E106 pointing at the opening marker.

7. Blank (Spaces)

Blanks, including spaces, tabs (\t), and newlines (\n), act as whitespace that separate adjacent tokens. They also allow indentation for readability. Statements are ended by a period.

8. Delimiters and Brackets

Symbol	Function
( )	Grouping expressions, function parameters and arguments
[ ]	Array and indexing
{}	Begins a code block
,	Separates items
.	Ends a simple statement (statement terminator)
" "	String literal boundary
' '	Character literal boundary

String escapes: \n (newline), \", \', and \\. Only straight quotes are accepted. Curly (smart) quotes cause

error E109 with a suggestion to replace them.

9. Fixed-and-Fix-Field Format

ClariFix uses a free-field format, so statements may begin at any column rather than at a fixed starting position. Spaces, tabs, and newlines separate tokens, and indentation is used only to improve readability. Because each simple statement ends with a period and code blocks are enclosed in { }, a statement can span several lines or share a line with others without changing its meaning.

10. Expressions

1. Arithmetic Expressions

Arithmetic expressions perform mathematical calculations using numbers, variables, and arithmetic operators. They follow operator precedence and parentheses when evaluating expressions.

- What they do: Calculate a numeric result.
- Examples:
  - 5 + 3 * 2: evaluates to 11.
  - 2 ^ 3: evaluates to 8.

2. Relational and Comparison Expressions

Relational expressions compare two values and return a Boolean result (true or false).

- What they do: Determine the relationship between two values.
- Examples:
  - x == 5: checks if x is equal to 5.
  - age >= 18: checks if age is greater than or equal to 18.

3. Logical Expressions

Logical expressions combine or modify Boolean conditions using logical operators.

- What they do: Evaluate conditions and return true or false.
- Logical Operators:
  - &&: Logical AND: returns true if both conditions are true.
  - ||: Logical OR: returns true if at least one condition is true.
  - !: Logical NOT: reverses a Boolean value.

- Examples:
  - x >= 6 && y == 7: true if x >= 6 and y == 7.
  - is_raining || is_snowing: true if it is raining or snowing.
  - !is_logged_in: returns true if the user is not logged in.

11. Data Types

Data Type	Description	Example
Integer	Whole numbers, positive or negative, no decimal point.	set age = 20.
Decimal	Numbers with a fractional part, stored as floating-point.	set price = 19.99.
Character	A single symbol enclosed in single quotes.	set grade = ‘A’.
Boolean	Holds only true or false	set isPassed = true.
String	A sequence of characters enclosed in double quotes.	set name = “Samantha”.
Arrays	An ordered, indexable collection of values enclosed in [ ].	set numbers = [1, 2, 3].
Other Abstract Data Types (Optional)	Not included — ClariFix intentionally limits its type system to these six to stay beginner-friendly.	-

12. Statements

Statement Type	Syntax / Rule	Example
Input Statements	ask <variable> “<prompt>” - this reads user input into a variable	ask(name). "*Enter your name*:"

Statement Type	Syntax / Rule	Example
Output Statements	show <expression> - displays a value or the result of an expression.	show("Hello, " + name.).
Assignment Statements	set <identifier> = <expression> (or set <identifier> to <expression>).	set total = price * quantity.

Conditional Statements	if <condition>: block, optional elif <condition>: and else: blocks, using indentation.	if (score >= 75){
		show ("Passed").
		else
		show ("Failed").
		}

Iterative Statements	while <condition>: for condition-based loops; for <var> in <range/list>: for counted or collection loops.	while(count <= 5) {
		show(count).
		set count to count + 1.
		}

Declaration Statements (Optional)	Not required separately, a variable is declared implicitly the first time it's used in a set statement (dynamic typing, like Python).	int score = 90. declares and assigns score in one step

The ClariFix Error System

This is the core contribution of the language.

Message Format

Every diagnostic uses the same layout and answers four questions: what, where, why, and how to fix

```
Error E201 (Name Error) at line 4, column 6
  show totl.
       ^^^^
What happened: The variable "totl" has not been created yet.
Why: Variables must be given a value with set before they are used.
How to fix: Did you mean "total"? Otherwise create it first: set totl = <value>.
```

Error Taxonomy

Category	Code range	Examples
Syntax	E1xx	E100 wrong file type; E101 missing colon; E102 unclosed quote or bracket; E103 illegal character; E104 identifier too long; E105 misplaced noise word; E106 unclosed

Category	Code range	Examples
		comment; E107 tab in indentation; E108 bad indentation; E109 curly quotes; E110 name starting with a digit; E111 unknown string escape; E112 bad character constant; E113 missing period at end of statement
Name	E2xx	E201 undefined variable; E202 reserved word used as identifier; E203 undefined function; E204 wrong argument count
Type	E3xx	E301 text + number; E302 ordering comparison of mismatched types; E303 non-Boolean condition
Runtime	E4xx	E401 division by zero; E402 failed conversion; E403 use of a missing return value; E404 list index out of range
Warning	W5xx	W501 = used in a condition; W502 loop condition never changes; W503 unreachable code after return; W504 variable set but never used
Hint	H6xx	H601 off-by-one risk in range; H602 comparing different types

Severity and Reporting Policy

- **Error:** the program cannot run (or stops, if raised at runtime).
- **Warning:** the program runs, but the code is probably wrong.
- **Hint:** informational advice that never blocks execution.
- Lexer and parser errors use line-level recovery (skip to the next line and continue), so up to 5 syntax errors are reported in one run. This avoids overwhelming beginners with a cascade. After 5, the message "Fix these first, then run again" is shown.
- Name and type errors found statically are all reported before execution. Runtime errors stop the program at the first occurrence.

Common-Mistake Detection Rules

Rule	Detected by
= used where == was intended in a condition (if x = 5:)	Parser
Missing : after if, elif, else, while, for, function, or missing . at the end of a simple statement	Parser
Mixed or tab indentation	Lexer
Unclosed quote, bracket, or comment	Lexer
Misspelled variable or function: "did you mean" using edit distance (Damerau–Levenshtein ≤ 2, ≤ 1 for names under 5 letters)	Semantic checker
Variable used before its first set	Semantic checker
"text" + number	Semantic checker (literals) / Runtime
Off-by-one: range(1, n) used where n itself seems intended (hint only)	Semantic checker
Code after return, break or continue that can never run	Semantic checker
Loop whose condition variables are never changed in the body	Semantic checker
Division by zero, bad list index, failed conversion	Runtime

Which Phase Catches What

Phase	Responsibility
Lexer	File type check, illegal characters, quotes, comments, identifier length, tabs, noise-word dropping, symbol table output
Parser	Missing colons and periods, block structure, = in conditions, misplaced noise words
Semantic checker	Undefined names, reserved-word identifiers, argument counts, literal type mismatches, warnings and hints
Runtime	Division by zero, conversions, list bounds, type errors that depend on values

Side-by-Side Comparison with Python

Mistake 1: adding text and a number
Python:

```
Traceback (most recent call last):
  File "main.py", line 2, in <module>
    print("Age: " + 5)
TypeError: can only concatenate str (not "int") to str
```

```
ClariFix:
Error E301 (Type Error) at line 2, column 6
  show "Age: " + 5.
       ^^^^^^^^^^^
```

What happened: You tried to add text ("Age: ") and a number (5).
Why: ClariFix does not mix text and numbers automatically.
How to fix: Convert the number first: show "Age: " + to_text(5).

Mistake 2: dividing by zero

```
Python:
ZeroDivisionError: division by zero
```

```
ClariFix:
Error E401 (Runtime Error) at line 7, column 14
  set avg = total / count.
                    ^^^^^
```

What happened: The program divided by zero because "count" is 0.
Why: Dividing by zero has no answer.
How to fix: Check the value first: if count > 0: ...
The difference is that ClariFix always states the cause in plain words, points at the exact expression, and gives a next step, in the same layout every time.

Formal Grammar

Core subset in EBNF:

```
program     = { statement } ;
block       = NEWLINE INDENT statement { statement } DEDENT ;
statement   = set_stmt | show_stmt | ask_stmt | if_stmt | while_stmt
            | for_stmt | func_def | return_stmt | "break" "." | "continue" "."
            | expr "." ;
set_stmt    = "set" target ( "=" | "to" ) expr "." ;
target      = IDENT [ "[" expr "]" ] ;
show_stmt   = "show" expr "." ;
ask_stmt    = "ask" IDENT STRING "." ;
if_stmt     = "if" expr [ "then" ] ":" block
              { "elif" expr [ "then" ] ":" block }
              [ "else" ":" block ] ;
while_stmt  = "while" expr ":" block ;
for_stmt    = "for" IDENT "in" expr ":" block ;
func_def    = "function" IDENT "(" [ params ] ")" ":" block ;
params      = IDENT { "," IDENT } ;
return_stmt = "return" [ expr ] "." ;
expr        = or_expr ;
or_expr     = and_expr { "or" and_expr } ;
and_expr    = not_expr { "and" not_expr } ;
not_expr    = "not" not_expr | equality ;
equality    = relation { ( "==" | "!=" | "is" [ "not" ] ) relation } ;
```

```
relation    = sum { ( "<" | ">" | "<=" | ">=" ) sum } ;
sum         = term { ( "+" | "-" ) term } ;
term        = unary { ( "*" | "/" | "//" | "%" ) unary } ;
unary       = "-" unary | power ;
power       = postfix [ "^" unary ] ;
postfix     = primary { "(" [ args ] ")" | "[" expr "]" } ;
args        = expr { "," expr } ;
primary     = INT | DECIMAL | STRING | CHAR | "true" | "false"
            | IDENT | "(" expr ")" | "[" [ args ] "]" ;
```

Token classes for the lexer:

Token	Pattern
IDENT	[A-Za-z_][A-Za-z0-9_]* (max 32, not reserved)
INT	[0-9]+
DECIMAL	[0-9]+ "." [0-9]+
STRING	a double quote, any characters except an unescaped double quote, then a double quote
CHAR	a single quote, exactly one character or escape, then a single quote
KEYWORD / RESERVED	the lowercase words listed in the Keywords and Reserved Words section
OPERATOR / DELIMITER	symbols listed in the Operation Symbols and Delimiters sections
NEWLINE, INDENT, DEDENT	generated from line breaks and 4-space indentation

Noise words, comments, and blanks are discarded by the lexer.

Sample Programs

Correct Program

```
# Class average calculator
function average(total, count):
    return total / count.

set total to 0.
set count to 0.
ask answer "How many scores? ".
set n = to_int(answer).
for i in range(0, n):
    ask entry "Enter score: ".
    set total = total + to_decimal(entry).
    set count = count + 1.
```

```
if count > 0 then:
    set avg = average(total, count).
    show "Average: " + to_text(avg).
else:
    show "No scores entered.".
```

Same Program with Deliberate Mistakes

```
function average(total, count)
    return total / count.

set total to 0.
set count to 0.
for i in range(0, n):
    set count = count + 1.
if count = 0:
    show "Average: " + average(total, count).
```

Expected output:

```
Error E101 (Syntax Error) at line 1, column 30
  function average(total, count)
                               ^
```

What happened: A colon is missing at the end of the function line.
Why: A colon tells ClariFix that an indented block follows.
How to fix: Add ":" after the closing parenthesis: function average(total, count):

```
Error E201 (Name Error) at line 6, column 19
  for i in range(0, n):
                    ^
```

What happened: The variable "n" has not been created yet.
Why: Variables must be given a value with set before they are used.
How to fix: Create it first, for example: set n = 5.

```
Warning W501 at line 8, column 4
  if count = 0:
     ^^^^^^^^^
```

What happened: A single "=" stores a value. It does not compare.
Why: Conditions need a comparison.
How to fix: Use "==" (or "is"): if count == 0:

```
Error E301 (Type Error) at line 9, column 10
  show "Average: " + average(total, count).
       ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

What happened: You tried to add text and a number.
Why: ClariFix does not mix text and numbers automatically.
How to fix: Convert the number: to_text(average(total, count))

Limitations and Future Work

- Only six data types; no dictionaries, tuples, or user-defined types.

- No modules, classes, or file I/O.
- Error messages are written in English only. A localization layer is possible future work.
- The "did you mean" suggestions use edit distance and may occasionally suggest a wrong name.
- Future work: an interactive "explain this error" mode, a step-through debugger, and a user study comparing ClariFix and Python error comprehension among beginners.

References

- Van Rossum, G., et al. *The Python Language Reference*. Python Software Foundation. https://docs.python.org/3/reference/
- Marceau, G., Fisler, K., & Krishnamurthi, S. (2011). Mind your language: On novices' interactions with error messages. *Proceedings of Onward! 2011*, ACM.
- Becker, B. A., et al. (2019). Compiler error messages considered unhelpful: The landscape of text-based programming error message research. *ITiCSE-WGR 2019*, ACM.
- Sebesta, R. W. *Concepts of Programming Languages*. Pearson.
- Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. *Compilers: Principles, Techniques, and Tools*. Pearson.

- DECIDE THE ENTRY FIELDS (LEXEME, TOKEN TYPE, ATTRIBUTES) AND LOOKUP METHOD, SUCH AS A HASH TABLE

Field	Description	Example
`id`	Running index, assigned on first insert	`3`
`lexeme`	Exact text, case-sensitive	`total`
`token_type`	Token class from your token table	`IDENTIFIER`, `CONST_INT`
`category`	Rubric category, for grouping	`Identifier`, `Constant`
`value`	Converted value (constants only)	`10`, `19.99`, `'A'`
`length`	Character count	`5`
`first_line`, **`first_col`**	Where it first appears	`2`, `5`

- LOOKUP METHOD

Method:	Hash table with separate chaining, implemented manually in C, since C has no built-in dictionary.
Hash function:	djb2. It computes a number from the characters of the lexeme, then takes `hash % 211` to get a bucket index.
Table size:	211 buckets (a prime number, which spreads entries more evenly). The size is fixed.
Collision handling:	Separate chaining. If two lexemes land in the same bucket, they are stored in a linked list in that bucket.
Key:	`(lexeme, token_type)`. Matching is case-sensitive, so `Total` and `total` are different

	entries.
Reserved table:	A separate fixed array of keywords, reserve words, and noise words (`set`, `show`, `break`, `to`, and so on). The lexer checks it first, before the symbol table. `true` and `false` are not in this table. They are scanned as `CONST_BOOLEAN` and inserted into the symbol table like other constants.

How a lookup works:

1. The lexer reads a lexeme.
2. It checks the reserved table. If found, the token type is returned and nothing goes into the symbol table.
3. Otherwise, the hash function computes the bucket index.
4. Only that bucket's list is searched.
5. If the entry is found, the existing entry is returned. If not, a new entry is created and inserted.

OPERATIONS

Function	Purpose
`symtab_init(st)`	Initialize an empty table
`symtab_lookup(st, lexeme, type)`	Return the entry if it exists, otherwise NULL
`symtab_insert(st, lexeme, type, value, line, col)`	Insert a new entry, or return the existing one if found
`symtab_record_token(line, col, lexeme, type)`	Add a token to the full token list
`symtab_record_invalid(line, col, lexeme, errcode)`	Record an unrecognized token
`symtab_write_file(st, src_name, out_path)`	Write the Symbol Table output text file
`symtab_free(st)`	Free all memory

- Fix the layout of the Symbol Table output text file (rubric 11).

Sections:

- **[A] Token List:** every token in order of appearance (shows keywords, reserve words, noise words, operators, delimiters, comments)
- **[B] Symbol Table:** unique identifiers and constants only, with their attributes
- **[C] Invalid Tokens:** lexemes that match no rule (rubric 10)
- **[D] Summary:** count per category

Sample Output

- **Columns are fixed-width and left-aligned**
- **First Seen is written as line:column**
- **Empty sections print (none)**

```
Sample input
# avg calc
set total to 10.
show total.
```

```
================================================================
                CLARIFIX SYMBOL TABLE
================================================================
Source file : sample.clx
----------------------------------------------------------------

[A] TOKEN LIST (in order of appearance)
No.  Line  Col  Lexeme       Token Type           Category
---  ----  ---  -----------  -------------------  ---------------
1    1     1    # avg calc   SINGLE_COMMENT       Comment
2    2     1    set          SET                  Keyword
3    2     5    total        IDENTIFIER           Identifier
4    2     11   to           NOISE                Noise Word
5    2     14   10           CONST_INT            Constant
6    2     16   .            STATEMENT_TERMINATOR Delimiter
7    3     1    show         SHOW                 Keyword
8    3     6    total        IDENTIFIER           Identifier
9    3     11   .            STATEMENT_TERMINATOR Delimiter

[B] SYMBOL TABLE (unique identifiers and constants)
ID  Lexeme  Token Type  Value  Length  First Seen
--  ------  ----------  -----  ------  ----------
1   total   IDENTIFIER  -      5       2:5
2   10      CONST_INT   10     2       2:14

[C] INVALID / UNRECOGNIZED TOKENS
Line  Col  Lexeme  Error  Reason
----  ---  ------  -----  ------
(none)

[D] SUMMARY
Identifiers: 2 | Keywords: 2 | Noise words: 1 | Constants: 1
Delimiters: 2 | Comments: 1 | Total tokens: 9
================================================================
```